



Instituto Politécnico de Viseu

Escola Superior de Tecnologia e Gestão de Viseu

Departamento de Informática

Unidade Curricular: Base de Dados II

Relatório do Projeto: Stand Carros e Oficina

Realizado por: Guilherme Bento – 25193

Tiago Rebelo - 25216

Lucas Santos - 27440

Viseu, 2026

Instituto Politécnico de Viseu
Escola Superior de Tecnologia e Gestão de Viseu
Departamento de Informática

Relatório relativo ao Projeto Prático

Curso de Licenciatura em Engenharia Informática

Unidade Curricular de Base de Dados II

Ano Letivo 2025/26

Viseu, 2026

ÍNDICE

Tema e Objetivos do Projeto 7

Tema 7

Objetivo da Aplicação 7

Tecnologias Utilizadas 7

Requisitos Funcionais Principais 7

Tipos de Utilizadores 7

Modelo de Dados 8

PostgreSQL 12

Tabelas da Base de Dados 12

Indexes das Tabelas: 14

Utilizadores e Permissões do SGBD 15

Utilizadores Criados 15

Procedures 17

Restantes Procedures: 19

Views 21

Restantes Views: 22

Todos os Indexes que usamos nas Views Materializadas: 24

Triggers 24

Restantes Triggers: 25

UDFs 26

Restantes UDFs: 28

Mongo DB 29

Os ficheiros que usamos: 29

Frontend 32

Tipos de Utilizadores da Aplicação: 32
Estrutura da Aplicação Web 32
Páginas de Autenticação 32
Páginas Principais 33
Páginas de Gestão de Clientes 35
Páginas de Reservas e Vendas 36
Páginas de Oficina e Serviços 37
Páginas de Recursos Humanos 40
Páginas de Sistema 41

Índice de Figuras

Figura 1- Modelo Concetual 8

Figura 2- Modelo Físico 9

Figura 3- Database Diagram 10

Figura 4- Database Diagram (continuação) 11

Figura 5-Indexes das Tabelas 14

Figura 6-Indexes das Tabelas (continuação) 14

Figura 7- Gerente 15

Figura 8-Funcionário 15

Figura 9- Mecânico 16

Figura 10- Cliente 16

Figura 11-Inserir_cliente 17

Figura 12- Inserir_carro 18

Figura 13- Inserir_carro(continuação) 18

Figura 14-criar_os 19

Figura 15-vw_avaliacoes_carros : Materializada 21

Figura 16-vw_avaliaçoes_detalhes : Simples 21

Figura 17-vw_carros_disponiveis: Simples 22

Figura 18-Indexes 24

Figura 19-fn_validar_email_cliente 24

Figura 20-fn_vendas_after_delete 25

Figura 21-fn_pecas_stock_alert 25

Figura 22-fn_exportar_relatorio_financeiro 26

Figura 23-fn_exportar_ordens_servico 27

Figura 24-fn_exportar_ordens_servico(continuação) 27
Figura 25-fn_exportar_inventario_carros 28
Figura 26-avalicoes_detalhes.json 29
Figura 27-diagnosticos_oficina.json 30
Figura 28- diagnosticos_oficina.json (continuação) 30
Figura 29-eventos_integracao.json 31
Figura 30-historico_manutencao.json 31
Figura 31 - historico_manutencao.json(continuação) 32
Figura 32 - Login 33
Figura 34 – Início 34
Figura 35 – Carros 34
Figura 36 - Clientes 35
Figura 37 – Reservas 36
Figura 38 – Vendas 37
Figura 39 – Oficina parte 1 38
Figura 40- Oficina parte 2 38
Figura 41 - Oficina parte 3 39
Figura 42 - Oficina parte 4 39
Figura 43 - Ordens de Serviço 40
Figura 44 – Avaliações 41
Figura 45 - Import/Export 42

Tema e Objetivos do Projeto

Tema

O tema escolhido para este projeto é um Sistema de Gestão de Stand de Automóveis e Oficina. A aplicação visa centralizar todas as operações de um stand, desde a aquisição de veículos até à venda final, integrando também um módulo de oficina para manutenção e serviços pós-venda.

Objetivo da Aplicação

O propósito do sistema é otimizar a gestão de inventário de veículos, o acompanhamento de clientes e o fluxo de trabalho da oficina. A aplicação resolve problemas de dispersão de informação, permitindo que gerentes, vendedores e mecânicos trabalhem numa plataforma unificada. Os utilizadores podem gerir reservas de carros, processar vendas, abrir ordens de serviço e controlar o stock de peças.

Tecnologias Utilizadas

Foram utilizadas as seguintes tecnologias: PostgreSQL, MongoDB, Python e Django.

Requisitos Funcionais Principais

- **Gestão de Automóveis:** Registo detalhado de veículos (marca, modelo, chassi, quilometragem) e seu estado de disponibilidade.
- **Processamento de Vendas e Reservas:** Registo de transações comerciais e reservas temporárias de veículos por clientes.
- **Gestão de Oficina:** Abertura e acompanhamento de Ordens de Serviço (OS), atribuição de mecânicos e baias de trabalho.
- **Sistema de Avaliações:** Feedback dos clientes sobre vendas e serviços realizados.

Tipos de Utilizadores

Tipo	Permissões	Páginas Acessíveis
Administrador	Gestão total do sistema e utilizadores	Todas
Gerente	Gestão de stands, funcionários e relatórios	Dashboard, Inventário, Vendas, Oficina
Funcionário	Gestão de clientes, reservas e vendas	Inventário, Clientes, Vendas, Reservas
Mecânico	Gestão de ordens de serviço e peças	Oficina, Peças, Ordens de Serviço
Cliente	Consulta de catálogo e histórico próprio	Catálogo, Reservas, Perfil

Modelos de Dados

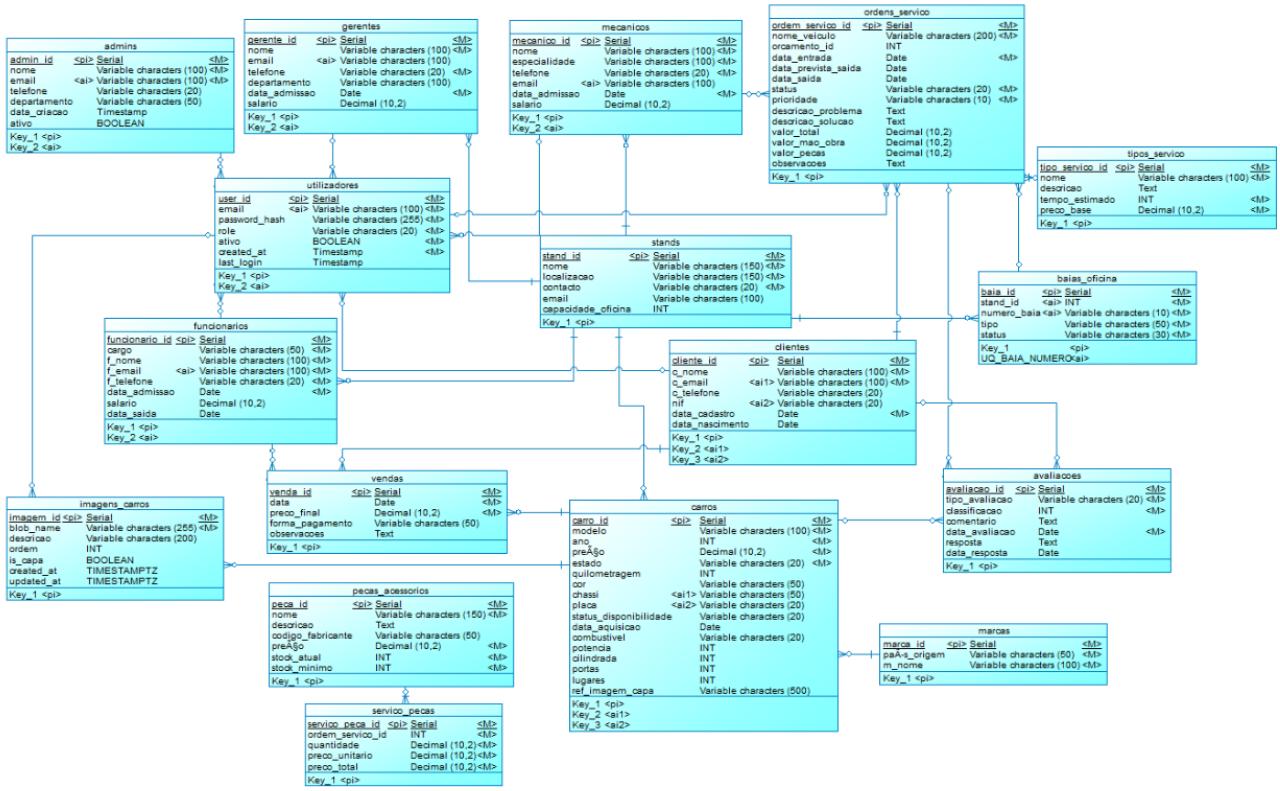


Figura 1- Modelo Concetual

Mostra entidades principais (stands, carros, clientes, vendas) e suas relações de forma abstrata. No sistema, serve de base para o modelo físico, identificando atributos essenciais para gestão de stand e oficina.

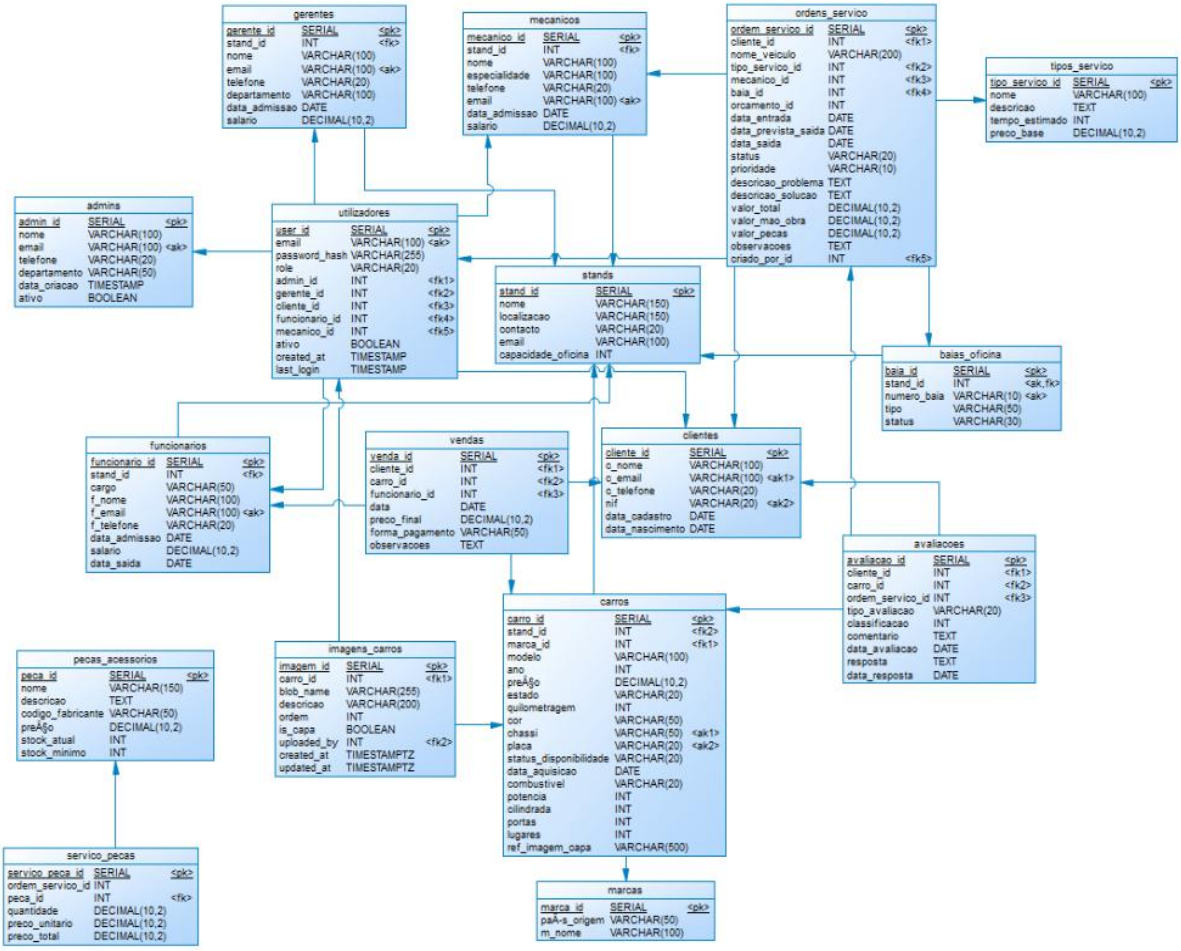


Figura 2- Modelo Físico

Representa o nosso diagrama de relações entre entidades da base de dados PostgreSQL, mostra chaves primárias, estrangeiras e tipos de dados. No nosso sistema, define como os dados de stands, carros, clientes e vendas estão interligados para suportar operações como inventário e vendas.

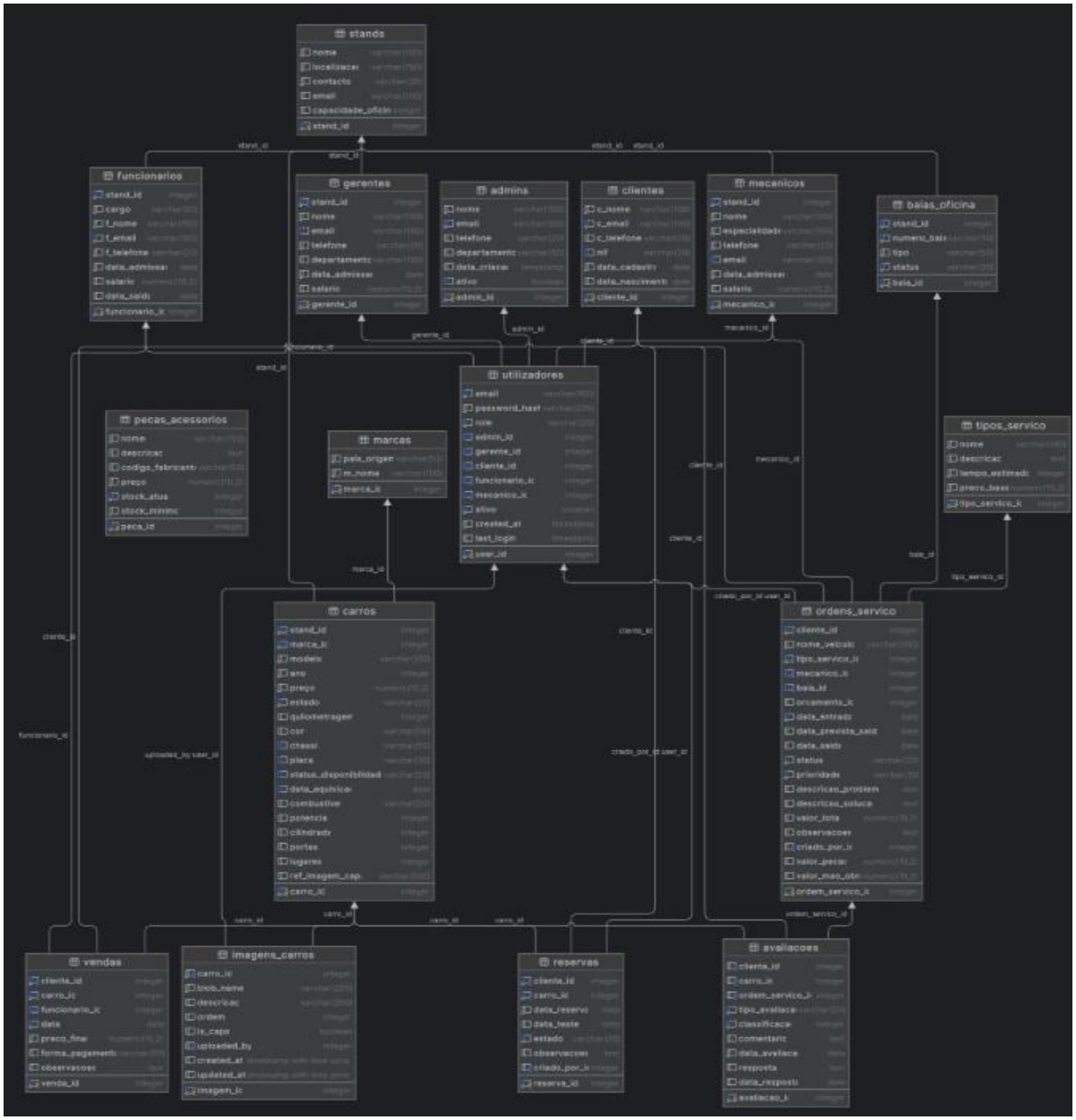


Figura 3- Database Diagram

Exibe visualmente as tabelas e ligações da base PostgreSQL. No nosso sistema, ilustra a estrutura completa para consultas eficientes em vendas, reservas e ordens de serviço.

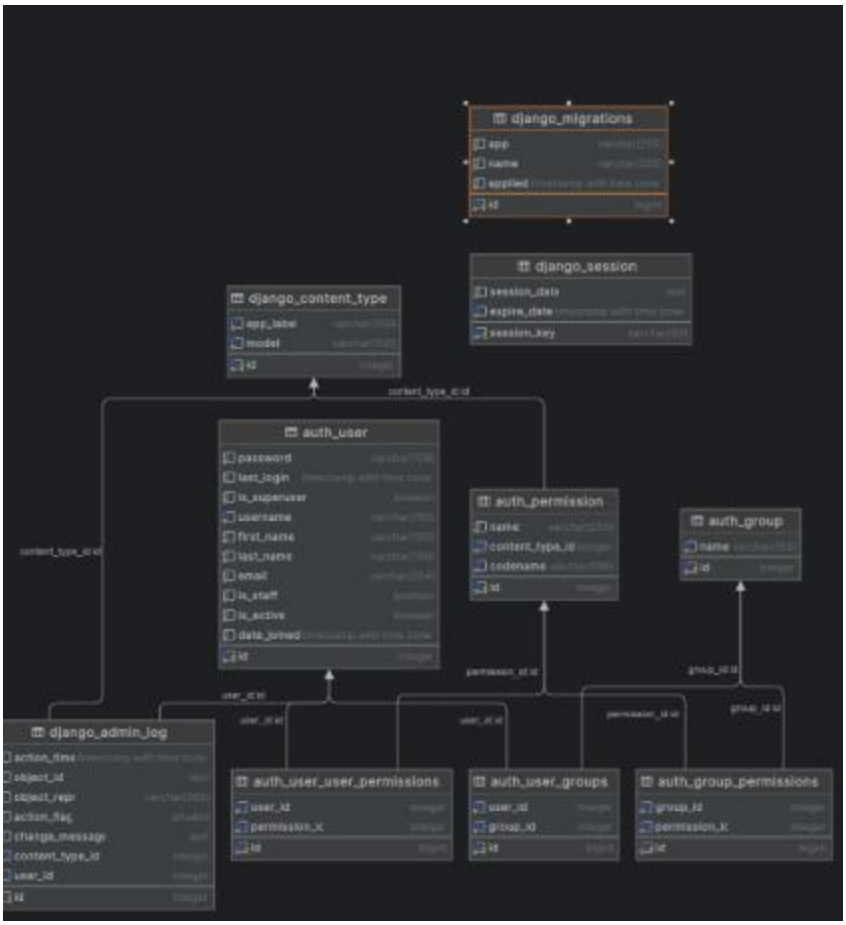


Figura 4- Database Diagram (continuação)

Continuação do diagrama anterior com tabelas adicionais como imagens e avaliações. No nosso sistema, completa a visão da BD para funcionalidades de galeria de carros e feedback de clientes.

PostgreSQL

Tabelas da Base de Dados

Nome Tabela	Objetivo	PK	FK Principal
stands	Locais físicos	stand_id	-
marcas	Catálogo de marcas de carros	marca_id	-
pecas_acessorios	Stock de peças e acessórios	peca_id	-
tipos_servico	Tipos de serviços de oficina	tipo_servico_id	-
carros	Inventário de veículos	carro_id	stand_id
baias_oficina	Baias físicas da oficina	baia_id	stand_id
funcionarios	Registo de funcionários	funcionario_id	stand_id
gerentes	Registo de gerentes de stand	gerente_id	stand_id
mecanicos	Registo de mecânicos	mecanico_id	stand_id

Nome Tabela	Objetivo	PK	FK Principal
admins	Administradores do sistema	admin_id	-
clientes	Registo de clientes	cliente_id	-
utilizadores	Autenticação e roles unificados	user_id	[admin, gerente, cliente, funcionario, mecanico]
imagens_carros	Galeria de fotos dos carros	imagem_id	carro_id, user_id
reservas	Reservas de carros por clientes	reserva_id	carro_id, cliente_id
vendas	Transações de venda	venda_id	carro_id, cliente_id
ordens_servico	Gestão de oficina	ordem_servico_id	cliente_id, tipo_servico_id
servico_pecas	Peças usadas em ordens de serviço	servico_peca_id	ordem_servico_id, peca_id
avaliacoes	Feedback de clientes	avaliacao_id	cliente_id, carro_id

Indexes das Tabelas:

```sql
-- ========================================
-- INDEXES
-- ========================================

-- carros
CREATE INDEX idx_carros_stand ON carros(stand_id);
CREATE INDEX idx_carros_marca ON carros(marca_id);
CREATE INDEX idx_carros_estado ON carros(estado);
CREATE INDEX idx_carros_status ON carros(status_disponibilidade);
CREATE INDEX idx_carros_data_aquisicao ON carros(data_aquisicao);

-- funcionarios
CREATE INDEX idx_funcionarios_stand ON funcionarios(stand_id);

-- gerentes
CREATE INDEX idx_gerentes_stand ON gerentes(stand_id);
CREATE INDEX idx_gerentes_email ON gerentes(email);

-- mecanicos
CREATE INDEX idx_mecanicos_stand ON mecanicos(stand_id);

-- admins
CREATE INDEX idx_admins_email ON admins(email);
CREATE INDEX idx_admins_ativo ON admins(ativo);

-- utilizadores
CREATE INDEX idx_utilizadores_email ON utilizadores(email);
CREATE INDEX idx_utilizadores_role ON utilizadores(role);
CREATE INDEX idx_utilizadores_ativo ON utilizadores(ativo);
CREATE INDEX idx_utilizadores_admin_id ON utilizadores(admin_id);
CREATE INDEX idx_utilizadores_gerente_id ON utilizadores(gerente_id);
CREATE INDEX idx_utilizadores_cliente_id ON utilizadores(cliente_id);
CREATE INDEX idx_utilizadores_funcionario_id ON utilizadores(funcionario_id);
CREATE INDEX idx_utilizadores_mecanico_id ON utilizadores(mecanico_id);

-- reservas
CREATE INDEX idx_reservas_estado ON reservas(estado);
CREATE INDEX idx_reservas_cliente ON reservas(cliente_id);
CREATE INDEX idx_reservas_carro ON reservas(carro_id);
```

Figura 5-Indexes das Tabelas

Índices em carros, funcionários, gerentes, admins e utilizadores otimizam buscas por stand, estado e email. Melhoram performance em listagens de inventário e autenticação de utilizadores.

```sql
-- vendas
CREATE INDEX idx_vendas_cliente ON vendas(cliente_id);
CREATE INDEX idx_vendas_data ON vendas(data);
CREATE INDEX idx_vendas_funcionario ON vendas(funcionario_id);
CREATE INDEX idx_vendas_carro ON vendas(carro_id);

-- ordens_servico
CREATE INDEX idx_ordens_status ON ordens_servico(status);
CREATE INDEX idx_ordens_cliente ON ordens_servico(cliente_id);
CREATE INDEX idx_ordens_mecanico ON ordens_servico(mecanico_id);
CREATE INDEX idx_ordens_tipo_servico ON ordens_servico(tipo_servico_id);
CREATE INDEX idx_ordens_baia ON ordens_servico(baia_id);
CREATE INDEX idx_ordens_prioridade ON ordens_servico(prioridade);
CREATE INDEX idx_ordens_data_entrada ON ordens_servico(data_entrada);

-- avaliacoes
CREATE INDEX idx_avaliacoes_tipo ON avaliacoes(tipo_avaliacao);
CREATE INDEX idx_avaliacoes_classificacao ON avaliacoes(classificacao);

-- pecas_acessorios
CREATE INDEX idx_pecas_stock ON pecas_acessorios(stock_atual);

-- baias_oficina
CREATE INDEX idx_baias_status ON baias_oficina(status);
```

Figura 6-Indexes das Tabelas (continuação)

Índices em vendas, ordens de serviço, avaliações e peças aceleram consultas por cliente, data e stock. Otimizam relatórios de oficina e controlo de inventário

Utilizadores e Permissões do SGBD

Utilizadores Criados

Admin

O admin é criado por omissão e tem acesso completo à Base de Dados.

```
-- ROLE: GERENTE (Acesso quase completo)
CREATE ROLE role_gerente LOGIN PASSWORD 'gerente';

GRANT CONNECT ON DATABASE "bdII_25193" TO role_gerente;
GRANT USAGE ON SCHEMA public TO role_gerente;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO role_gerente;

GRANT INSERT, UPDATE, DELETE ON
    carros, clientes, funcionarios, mecanicos,
    reservas, vendas, ordens_servico, avaliacoes,
    imagens_carros, baias_oficina, servico_pecas
TO role_gerente;

GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO role_gerente;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO role_gerente;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO role_gerente;
```

Figura 7- Gerente

Permite ao gerente aceder a dashboard, inventário, vendas e oficina para gestão global.

```
-- ROLE: FUNCIONARIO (Vendas e Atendimento)
CREATE ROLE role_funcionario LOGIN PASSWORD 'funcionario';

GRANT CONNECT ON DATABASE "bdII_25193" TO role_funcionario;
GRANT USAGE ON SCHEMA public TO role_funcionario;

GRANT SELECT ON
    stands, marcas, carros, clientes,
    funcionarios, pecas_acessorios, tipos_servico,
    reservas, vendas, imagens_carros,
    ordens_servico, baias_oficina, servico_pecas
TO role_funcionario;

GRANT INSERT, UPDATE ON
    clientes, reservas, vendas
TO role_funcionario;

GRANT UPDATE (status_disponibilidade) ON carros TO role_funcionario;

GRANT USAGE, SELECT ON
    clientes_cliente_id_seq, reservas_reserva_id_seq, vendas_venda_id_seq
TO role_funcionario;
```

Figura 8-Funcionário

No nosso sistema, restringe acesso a inventário, clientes, vendas e reservas, adequado para vendedores.

```sql
-- ROLE: MECANICO (Oficina)
CREATE ROLE role_mecanico LOGIN PASSWORD 'mecanico';

GRANT CONNECT ON DATABASE "bdII_25193" TO role_mecanico;
GRANT USAGE ON SCHEMA public TO role_mecanico;

GRANT SELECT ON
    stands, marcas, carros, clientes, mecanicos,
    tipos_servico, pecas_acessorios,
    baias_oficina, ordens_servico, imagens_carros, servico_pecas
TO role_mecanico;

GRANT INSERT, UPDATE ON
    ordens_servico
TO role_mecanico;

GRANT INSERT, UPDATE ON
    servico_pecas
TO role_mecanico;

GRANT UPDATE (status_disponibilidade) ON carros TO role_mecanico;
GRANT UPDATE (status) ON baias_oficina TO role_mecanico;

GRANT USAGE, SELECT ON
    ordens_servico_ordem_servico_id_seq
TO role_mecanico;

GRANT USAGE, SELECT ON
    servico_pecas_servico_peca_id_seq
TO role_mecanico;
```

Figura 9- Mecânico

No nosso sistema, o mecânico, foca-se na oficina, peças e ordens de serviço para execução de manutenções.

```sql
-- ROLE: CLIENTE (Portal - Read-only + ações limitadas)
CREATE ROLE role_cliente LOGIN PASSWORD 'cliente';

GRANT CONNECT ON DATABASE "bdII_25193" TO role_cliente;
GRANT USAGE ON SCHEMA public TO role_cliente;

GRANT SELECT ON
    stands, marcas, carros, imagens_carros,
    tipos_servico, pecas_acessorios
TO role_cliente;

GRANT INSERT ON reservas, avaliacoes TO role_cliente;

GRANT USAGE ON
    reservas_reserva_id_seq,
    avaliacoes_avaliacao_id_seq
TO role_cliente;
```

Figura 10- Cliente

No nosso sistema, limita a catálogo, reservas e perfil para consultas e ações de clientes.

Procedures

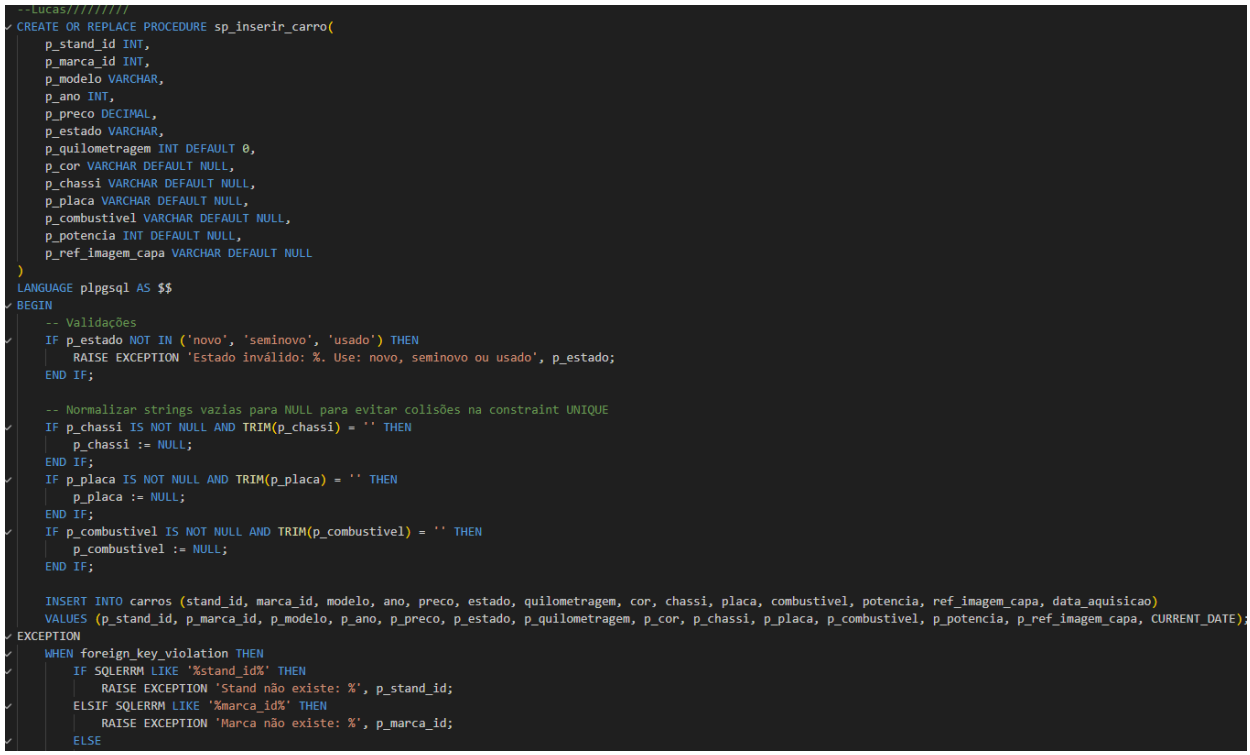
Três exemplos do nosso trabalho:

```
--Guilherme///////
CREATE OR REPLACE PROCEDURE sp_inserir_cliente(
    p_nome VARCHAR,
    p_email VARCHAR,
    p_telefone VARCHAR,
    p_nif VARCHAR,
    p_data_nascimento DATE DEFAULT NULL
)
LANGUAGE plpgsql AS $$
BEGIN
    IF NOT fn_validar_nif(p_nif) THEN
        RAISE EXCEPTION 'NIF inválido: %', p_nif;
    END IF;

    INSERT INTO clientes (c_nome, c_email, c_telefone, nif, data_nascimento)
    VALUES (p_nome, p_email, p_telefone, p_nif, p_data_nascimento);
EXCEPTION
    WHEN unique_violation THEN
        IF SQLERRM LIKE '%c_email%' THEN
            RAISE EXCEPTION 'Já existe um cliente com este email: %', p_email;
        ELSIF SQLERRM LIKE '%nif%' THEN
            RAISE EXCEPTION 'Já existe um cliente com este NIF: %', p_nif;
        ELSE
            RAISE EXCEPTION 'Dados duplicados: %', SQLERRM;
        END IF;
END;
$$;
```

Figura 11-Inserir_cliente

No nosso sistema, insere cliente na BD com validação de dados essenciais como NIF e email.



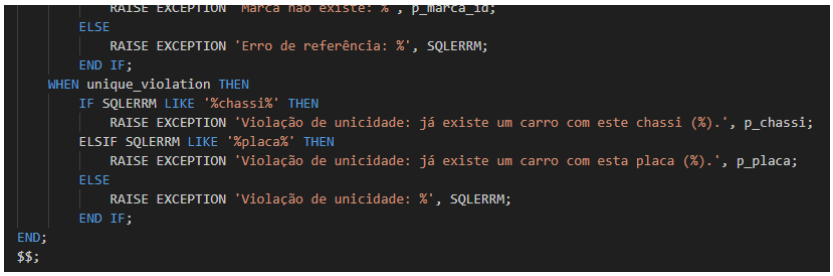
```
--Lucas/////////
CREATE OR REPLACE PROCEDURE sp_inserir_carro(
    p_stand_id INT,
    p_marca_id INT,
    p_modelo VARCHAR,
    p_ano INT,
    p_preco DECIMAL,
    p_estado VARCHAR,
    p_quilometragem INT DEFAULT 0,
    p_cor VARCHAR DEFAULT NULL,
    p_chassi VARCHAR DEFAULT NULL,
    p_placa VARCHAR DEFAULT NULL,
    p_combustivel VARCHAR DEFAULT NULL,
    p_potencia INT DEFAULT NULL,
    p_ref_imagem_capa VARCHAR DEFAULT NULL
)
LANGUAGE plpgsql AS $$
BEGIN
    -- Validações
    IF p_estado NOT IN ('novo', 'seminovo', 'usado') THEN
        RAISE EXCEPTION 'Estado inválido: %. Use: novo, seminovo ou usado', p_estado;
    END IF;

    -- Normalizar strings vazias para NULL para evitar colisões na constraint UNIQUE
    IF p_chassi IS NOT NULL AND TRIM(p_chassi) = '' THEN
        p_chassi := NULL;
    END IF;
    IF p_placa IS NOT NULL AND TRIM(p_placa) = '' THEN
        p_placa := NULL;
    END IF;
    IF p_combustivel IS NOT NULL AND TRIM(p_combustivel) = '' THEN
        p_combustivel := NULL;
    END IF;

    INSERT INTO carros (stand_id, marca_id, modelo, ano, preco, estado, quilometragem, cor, chassi, placa, combustivel, potencia, ref_imagem_capa, data_aquisicao)
    VALUES (p_stand_id, p_marca_id, p_modelo, p_ano, p_preco, p_estado, p_quilometragem, p_cor, p_chassi, p_placa, p_combustivel, p_potencia, p_ref_imagem_capa, CURRENT_DATE);
EXCEPTION
    WHEN foreign_key_violation THEN
        IF SQLERRM LIKE '%stand_id%' THEN
            RAISE EXCEPTION 'Stand não existe: %', p_stand_id;
        ELSIF SQLERRM LIKE '%marca_id%' THEN
            RAISE EXCEPTION 'Marca não existe: %', p_marca_id;
        ELSE
```

Figura 12- Inserir_carro

No nosso sistema, adiciona carro ao inventário com stand, marca e especificações técnicas.



```
            RAISE EXCEPTION 'Marca não existe: %', p_marca_id;
        ELSE
            RAISE EXCEPTION 'Erro de referência: %', SQLERRM;
        END IF;
    WHEN unique_violation THEN
        IF SQLERRM LIKE '%chassi%' THEN
            RAISE EXCEPTION 'Violação de unicidade: já existe um carro com este chassi (%).', p_chassi;
        ELSIF SQLERRM LIKE '%placa%' THEN
            RAISE EXCEPTION 'Violação de unicidade: já existe um carro com esta placa (%).', p_placa;
        ELSE
            RAISE EXCEPTION 'Violação de unicidade: %', SQLERRM;
        END IF;
END;
$$;
```

Figura 13- Inserir_carro(continuação)

No nosso sistema, inclui quilometragem, preço e estado para registo completo no inventário.

```sql
--Tiago
CREATE OR REPLACE PROCEDURE sp_criar_os(
    p_carro_id INT,
    p_cliente_id INT,
    p_tipo_servico_id INT,
    p_valor_mao_obra DECIMAL,
    p_mecanico_id INT DEFAULT NULL,
    p_baia_id INT DEFAULT NULL,
    p_data_prevista TIMESTAMP DEFAULT NULL,
    p_prioridade VARCHAR DEFAULT 'normal',
    p_diagnostico TEXT DEFAULT NULL,
    p_observacoes TEXT DEFAULT NULL
)
LANGUAGE plpgsql AS $$
DECLARE
    v_nome_veiculo VARCHAR := NULL;
BEGIN
    IF p_carro_id IS NOT NULL THEN
        SELECT modelo INTO v_nome_veiculo FROM carros WHERE carro_id = p_carro_id;
    END IF;

    IF v_nome_veiculo IS NULL THEN
        v_nome_veiculo := '---';
    END IF;

    INSERT INTO ordens_servico (cliente_id, nome_veiculo, tipo_servico_id, mecanico_id, baia_id, data_prevista_saida, prioridade, descricao_problema, observacoes, valor_total, criado_por_id)
    VALUES (p_cliente_id, v_nome_veiculo, p_tipo_servico_id, p_mecanico_id, p_baia_id, p_data_prevista, p_prioridade, p_diagnostico, p_observacoes, COALESCE(p_valor_mao_obra,0), NULL);
EXCEPTION
    WHEN foreign_key_violation THEN
        IF SQLERRM LIKE '%cliente_id%' THEN
            RAISE EXCEPTION 'Cliente não existe: %', p_cliente_id;
        ELSIF SQLERRM LIKE '%tipo_servico_id%' THEN
            RAISE EXCEPTION 'Tipo de serviço não existe: %', p_tipo_servico_id;
        ELSIF SQLERRM LIKE '%mecanico_id%' THEN
            RAISE EXCEPTION 'Mecânico não existe: %', p_mecanico_id;
        ELSIF SQLERRM LIKE '%baia_id%' THEN
            RAISE EXCEPTION 'Baia não existe: %', p_baia_id;
        ELSE
            RAISE EXCEPTION 'Erro de referência: %', SQLERRM;
        END IF;
END;
$$;
```

Figura 14-criar_os

No nosso sistema, abre OS com cliente, carro e detalhes para gestão de oficina

Restantes Procedures:

Procedure	Função
sp_eliminar_cliente	Remove cliente após verificar vendas/reservas procedures.sql
sp_atualizar_cliente	Atualiza dados do cliente procedures.sql
sp_eliminar_carro	Remove carro e dados associados (imagens, reservas) procedures.sql
sp_atualizar_carro	Atualiza especificações do carro procedures.sql
sp_atualizar_ref_imagem_capa	Define imagem de capa do carro procedures.sql

Procedure	Função
sp_limpar_ref_imagem_capa	Remove referência à imagem de capa procedures.sql
sp_registar_venda	Regista venda e atualiza status do carro procedures.sql
sp_eliminar_imagem_carro	Remove imagem e limpa referências procedures.sql
sp_inserir_imagem_carro	Adiciona imagem com controlo de capa procedures.sql
sp_atualizar_imagem_carro	Atualiza metadados da imagem procedures.sql
sp_atualizar_last_login	Regista último login do utilizador procedures.sql
sp_inserir_avaliacao	Adiciona avaliação (venda ou serviço) com validação de classificação 1-5 procedures.sql
sp_eliminar_avaliacao	Remove avaliação sem restrições procedures.sql
sp_eliminar_avaliacao_por_utilizador	Remove avaliação com controlo de permissões baseado no role (admin/gerente/funcionário podem eliminar qualquer; clientes apenas as suas) procedures.sql
Procedure	Descrição
sp_concluir_os	Finaliza ordem calculando valor total (peças + mão de obra) e define data de saída procedures.sql
sp_eliminar_os	Remove ordem de serviço apenas se status for concluída procedures.sql

Views

Três exemplos do nosso trabalho:

```sql
--Tiago
CREATE MATERIALIZED VIEW vw_avaliacoes_carros AS
SELECT
    a.carro_id,
    COUNT(a.avaliacao_id) AS total_avaliacoes,
    ROUND(AVG(a.classificacao)::numeric, 2) AS media_classificacao,
    MIN(a.classificacao) AS classificacao_minima,
    MAX(a.classificacao) AS classificacao_maxima,
    COUNT(CASE WHEN a.comentario IS NOT NULL AND a.comentario != '' THEN 1 END) AS avaliacoes_com_comentario,
    MAX(a.data_avaliacao) AS ultima_avaliacao
FROM avaliacoes a
WHERE a.tipo_avaliacao = 'venda' AND a.carro_id IS NOT NULL
GROUP BY a.carro_id
ORDER BY media_classificacao DESC, total_avaliacoes DESC;
```

Figura 15-vw_avaliacoes_carros : Materializada

No nosso sistema, otimiza consultas de avaliações agregadas para relatórios rápidos.

```sql
--Guilherme
CREATE OR REPLACE VIEW vw_avaliacoes_detalhes AS
SELECT
    a.avaliacao_id,
    a.cliente_id,
    c.c_nome AS cliente_nome,
    a.carro_id,
    CASE WHEN a.carro_id IS NOT NULL
        THEN m.m_nome || ' ' || ca.modelo || ' (' || ca.ano || ')'
    END AS carro_info,
    a.ordem_servico_id,
    CASE WHEN a.ordem_servico_id IS NOT NULL
        THEN ts.nome || ' - ' || os.nome_veiculo
    END AS servico_info,
    a.tipo_avaliacao,
    a.classificacao,
    a.comentario,
    a.data_avaliacao
FROM avaliacoes a
JOIN clientes c ON a.cliente_id = c.cliente_id
LEFT JOIN carros ca ON a.carro_id = ca.carro_id
LEFT JOIN marcas m ON ca.marca_id = m.marca_id
LEFT JOIN ordens_servico os ON a.ordem_servico_id = os.ordem_servico_id
LEFT JOIN tipos_servico ts ON os.tipo_servico_id = ts.tipo_servico_id
ORDER BY a.data_avaliacao DESC;
```

Figura 16-vw_avaliaçoes_detalhes : Simples

No nosso sistema, fornece detalhes de avaliações por cliente e carro para análise de feedback.

```sql
--Lucas
CREATE OR REPLACE VIEW vw_carros_disponiveis AS
SELECT
    c.carro_id,
    c.marca_id,
    c.stand_id,
    c.modelo,
    c.ano,
    c.preço AS preco,
    c.preço AS "preço",
    c.estado,
    c.quilometragem,
    c.cor,
    c.status_disponibilidade,
    c.data_aquisicao,
    c.cilindrada,
    c.portas,
    c.lugares,
    c.ref_imagem_capa
FROM carros c
WHERE c.status_disponibilidade = 'disponivel';
```

Figura 17-vw_carros_disponiveis: Simples

No nosso sistema, lista carros prontos para venda ou reserva no catálogo.

Restantes Views:

View	Descrição
vw_clientes_utilizadores	Relaciona clientes com utilizadores mostrando email, ID e nome views.sql
vw_avaliacoes_detalhes	Detalhes completos das avaliações com informações de cliente, carro e serviço views.sql
vw_vendas_completas	Vendas com dados completos de cliente, carro, marca e funcionário responsável views.sql
vw_carros_disponiveis	Lista carros com status "disponível" para venda views.sql
vw_carros_completos	Informação detalhada de todos os carros incluindo stand, marca e características técnicas views.sql
vw_imagens_carros_galeria	Galeria de imagens dos carros com URL, descrição e ordem de exibição views.sql
vw_reservas_detalhes	Reservas com detalhes do cliente, carro, marca e estado da reserva views.sql

View	Descrição
vw_funcionarios_vendas	Funcionários com total de vendas realizadas e valor total vendido views.sql
vw_stands_oficina	Lista stands que possuem oficina associada views.sql
vw_mecanicos_disponibilidade	Mecânicos com contagem de ordens de serviço ativas e disponibilidade views.sql
vw_ordem_servico_detalhes	Detalhes completos de ordens de serviço incluindo cliente, mecânico, baia e valores views.sql
vw_os_em_andamento	Ordens de serviço com status "em_andamento" para acompanhamento views.sql
vw_os_atrasadas	Ordens de serviço com data prevista de saída ultrapassada views.sql
vw_ordens_servico_concluidas	Histórico de ordens de serviço concluídas com todos os detalhes views.sql
vw_baias_disponiveis	Baias de oficina sem ordem de serviço ativa atribuída views.sql
vw_pecas_stock_baixo	Peças com quantidade em stock abaixo do mínimo definido views.sql
vw_servico_pecas	Peças utilizadas em cada ordem de serviço com quantidades e valores views.sql
vw_clientes_detalhes	Informação completa de clientes incluindo total gasto e número de compras views.sql

Todos os Indexes que usamos nas Views Materializadas:

```sql
-- ========================================
-- INDEXES PARA VIEWS MATERIALIZADAS
-- ========================================

CREATE UNIQUE INDEX idx_vw_clientes_compras_pk ON vw_clientes_compras(cliente_id);
CREATE INDEX idx_vw_clientes_compras_valor ON vw_clientes_compras(valor_total DESC);

CREATE UNIQUE INDEX idx_vw_avaliacoes_carros_pk ON vw_avaliacoes_carros(carro_id);
CREATE INDEX idx_vw_avaliacoes_carros_media ON vw_avaliacoes_carros(media_classificacao DESC);

CREATE UNIQUE INDEX idx_vw_avaliacoes_servicos_pk ON vw_avaliacoes_servicos(ordem_servico_id);
CREATE INDEX idx_vw_avaliacoes_servicos_media ON vw_avaliacoes_servicos(media_classificacao DESC);

CREATE UNIQUE INDEX idx_vw_funcionarios_desempenho_pk ON vw_funcionarios_desempenho(funcionario_id);

CREATE UNIQUE INDEX idx_vw_stock_atual_pk ON vw_stock_atual(stand_id);
CREATE INDEX idx_vw_stock_atual_disponiveis ON vw_stock_atual(disponiveis DESC);

CREATE UNIQUE INDEX idx_vw_mecanicos_desempenho_pk ON vw_mecanicos_desempenho(mecanico_id);

CREATE UNIQUE INDEX idx_vw_servicos_mais_vendidos_pk ON vw_servicos_mais_vendidos(tipo_servico_id);
CREATE INDEX idx_vw_servicos_mais_vendidos_faturacao ON vw_servicos_mais_vendidos(faturacao_total DESC);
```

Figura 18-Indexes

Lista de índices criados na BD PostgreSQL. No nosso sistema, acelera consultas frequentes em carros, vendas e ordens de serviço.

Triggers

Três exemplos do nosso trabalho:

```sql
--Guilherme
CREATE OR REPLACE FUNCTION fn_validar_email_cliente()
RETURNS TRIGGER AS $$
BEGIN
    --Valida formato de email
    IF NEW.c_email !~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}$' THEN
        RAISE EXCEPTION 'Email inválido: %', NEW.c_email;
    END IF;

    --Normaliza email (lowercase)
    NEW.c_email := LOWER(NEW.c_email);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tg_clientes_validar_email ON clientes;
CREATE TRIGGER tg_clientes_validar_email
BEFORE INSERT OR UPDATE ON clientes
FOR EACH ROW
EXECUTE FUNCTION fn_validar_email_cliente();
```

Figura 19-fn_validar_email_cliente

No nosso sistema, valida formato de email antes de inserir/atualizar cliente.

```sql
--Tiago
CREATE OR REPLACE FUNCTION fn_vendas_after_delete()
RETURNS TRIGGER AS $$
BEGIN
        UPDATE carros
        SET status_disponibilidade = 'disponivel'
        WHERE carro_id = OLD.carro_id
            AND status_disponibilidade = 'vendido';
        RETURN OLD;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tg_vendas_after_delete ON vendas;
CREATE TRIGGER tg_vendas_after_delete
AFTER DELETE ON vendas
FOR EACH ROW
EXECUTE FUNCTION fn_vendas_after_delete();
```

Figura 20-fn_vendas_after_delete

No nosso sistema, atualiza inventário após eliminação de venda.

```sql
CREATE OR REPLACE FUNCTION fn_pecas_stock_alert()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stock_atual < NEW.stock_minimo AND OLD.stock_atual >= OLD.stock_minimo THEN
        RAISE NOTICE 'ALERTA: Stock baixo para peça % (ID: %). Stock atual: %, Mínimo: %', NEW.nome, NEW.peca_id, NEW.stock_atual, NEW.stock_minimo;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS tg_pecas_stock_alert ON pecas_acessorios;
CREATE TRIGGER tg_pecas_stock_alert
AFTER UPDATE ON pecas_acessorios
FOR EACH ROW
EXECUTE FUNCTION fn_pecas_stock_alert();
```

Figura 21-fn_pecas_stock_alert

No nosso sistema, alerta stock baixo ao usar peças em ordens de serviço.

Restantes Triggers:

Trigger	Descrição
fn_carros_before_delete	Impede eliminação de carros que têm vendas associadas triggers.sql
fn_limitar_imagens_carro	Limita máximo de 20 imagens por carro triggers.sql
fn_imagens_carro_atualizar_capa	Atualiza imagem de capa garantindo apenas uma capa por carro triggers.sql
fn_vendas_before_insert	Verifica disponibilidade do carro antes de registar venda triggers.sql

Trigger	Descrição
fn_ordens_servico_status	Atualiza automaticamente status para concluída e prioridade para urgente quando aplicável triggers.sql
fn_reservas_before_insert	Valida disponibilidade do carro antes de criar reserva triggers.sql
fn_reservas_after_delete	Restaura status do carro para disponível após eliminar reserva triggers.sql

UDFs

Três exemplos do nosso trabalho:

```sql
--Guilherme
CREATE OR REPLACE FUNCTION fn_exportar_relatorio_financeiro(
    p_mes INT,
    p_ano INT
)
RETURNS TABLE (
    stand_nome VARCHAR,
    total_vendas_carros BIGINT,
    receita_vendas_carros NUMERIC,
    total_servicos_oficina BIGINT,
    receita_servicos_oficina NUMERIC,
    receita_total NUMERIC
) AS $$
BEGIN
    RETURN QUERY
    WITH vendas_mes AS (
        SELECT
            ca.stand_id,
            COUNT(v.venda_id) AS total_vendas,
            COALESCE(SUM(v.preco_final), 0) AS receita_vendas
        FROM vendas v
        INNER JOIN carros ca ON v.carro_id = ca.carro_id
        WHERE EXTRACT(MONTH FROM v.data) = p_mes
          AND EXTRACT(YEAR FROM v.data) = p_ano
        GROUP BY ca.stand_id
    ),
    servicos_mes AS (
        SELECT
            s.stand_id,
            0::BIGINT AS total_servicos,
            0::NUMERIC AS receita_servicos
        FROM stands s
    )
    SELECT
        s.nome,
        COALESCE(vm.total_vendas, 0),
        COALESCE(vm.receita_vendas, 0)::NUMERIC,
        COALESCE(sm.total_servicos, 0),
        COALESCE(sm.receita_servicos, 0)::NUMERIC,
        (COALESCE(vm.receita_vendas, 0) + COALESCE(sm.receita_servicos, 0))::NUMERIC AS receita_total
    FROM stands s
    LEFT JOIN vendas_mes vm ON s.stand_id = vm.stand_id
    LEFT JOIN servicos_mes sm ON s.stand_id = sm.stand_id
    ORDER BY receita_total DESC;
END;
$$ LANGUAGE plpgsql;
```

Figura 22-fn_exportar_relatorio_financeiro

No nosso sistema, gera relatório financeiro com totais de vendas e serviços.

```sql
--Tiago
CREATE OR REPLACE FUNCTION fn_exportar_ordens_servico(
    p_data_inicio DATE,
    p_data_fim DATE,
    p_status VARCHAR DEFAULT NULL
)
RETURNS TABLE (
    ordem_servico_id INT,
    cliente_nome VARCHAR,
    cliente_nif VARCHAR,
    carro_modelo VARCHAR,
    carro_placa VARCHAR,
    mecanico_nome VARCHAR,
    tipo_servico VARCHAR,
    status VARCHAR,
    prioridade VARCHAR,
    data_entrada TIMESTAMP,
    data_prevista TIMESTAMP,
    data_conclusao TIMESTAMP,
    valor_mao_obra NUMERIC,
    valor_pecas NUMERIC,
    valor_total NUMERIC,
    stand_nome VARCHAR
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        os.ordem_servico_id,
        c.c_nome,
        c.nif,
        os.nome_veiculo AS carro_modelo,
        NULL::VARCHAR AS carro_placa,
        me.nome,
        ts.nome,
        os.status,
        os.prioridade,
        os.data_entrada,
        os.data_prevista_saida,
        os.data_saida,
        0::NUMERIC AS valor_mao_obra,
        0::NUMERIC AS valor_pecas,
        COALESCE(os.valor_total, 0)::NUMERIC AS valor_total,
        s.nome
    FROM ordens_servico os
    INNER JOIN clientes c ON os.cliente_id = c.cliente_id
```

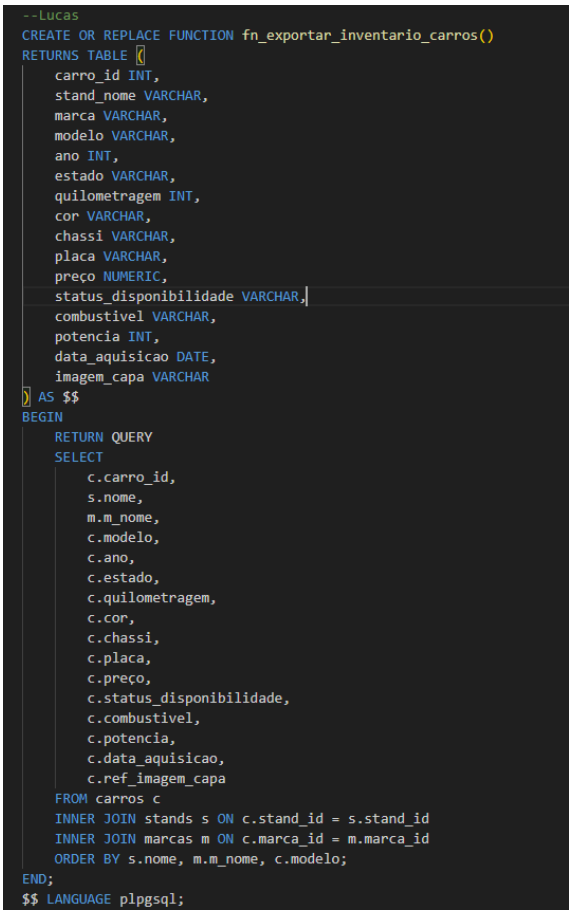
Figura 23-fn_exportar_ordens_servico

No nosso sistema, exporta lista de OS com detalhes para relatórios de oficina.

```sql
        s.nome
    FROM ordens_servico os
    INNER JOIN clientes c ON os.cliente_id = c.cliente_id
    LEFT JOIN mecanicos me ON os.mecanico_id = me.mecanico_id
    LEFT JOIN tipos_servico ts ON os.tipo_servico_id = ts.tipo_servico_id
    CROSS JOIN (SELECT NULL::VARCHAR AS nome) s
    WHERE os.data_entrada::DATE BETWEEN p_data_inicio AND p_data_fim
      AND (p_status IS NULL OR os.status = p_status)
    ORDER BY os.data_entrada DESC;
END;
$$ LANGUAGE plpgsql;
```

Figura 24-fn_exportar_ordens_servico(continuação)

No nosso sistema, inclui colunas adicionais como custos e status.



```sql
--Lucas
CREATE OR REPLACE FUNCTION fn_exportar_inventario_carros()
RETURNS TABLE (
    carro_id INT,
    stand_nome VARCHAR,
    marca VARCHAR,
    modelo VARCHAR,
    ano INT,
    estado VARCHAR,
    quilometragem INT,
    cor VARCHAR,
    chassi VARCHAR,
    placa VARCHAR,
    preço NUMERIC,
    status_disponibilidade VARCHAR,
    combustivel VARCHAR,
    potencia INT,
    data_aquisicao DATE,
    imagem_capa VARCHAR
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.carro_id,
        s.nome,
        m.m_nome,
        c.modelo,
        c.ano,
        c.estado,
        c.quilometragem,
        c.cor,
        c.chassi,
        c.placa,
        c.preço,
        c.status_disponibilidade,
        c.combustivel,
        c.potencia,
        c.data_aquisicao,
        c.ref_imagem_capa
    FROM carros c
    INNER JOIN stands s ON c.stand_id = s.stand_id
    INNER JOIN marcas m ON c.marca_id = m.marca_id
    ORDER BY s.nome, m.m_nome, c.modelo;
END;
$$ LANGUAGE plpgsql;
```

Figura 25-fn_exportar_inventario_carros

No nosso sistema, gera inventário completo de carros por stand.

Restantes UDFs:

Função	Descrição	Retorno
fn_autenticar_utilizador	Autentica utilizador por email/password e retorna dados completos incluindo role e IDs udfs.sql	TABLE
fn_carro_disponivel	Verifica se carro está disponível para venda/reserva udfs.sql	BOOLEAN
fn_validar_nif	Valida NIF português com verificação de dígito de controlo udfs.sql	BOOLEAN

Função	Descrição	Retorno
fn_reserva_disponivel	Verifica se carro está disponível para reserva num intervalo de datas udfs.sql	BOOLEAN
fn_baia_disponivel	Verifica disponibilidade de baia de oficina numa data/hora específica udfs.sql	BOOLEAN
fn_gerar_url_imagem	Gera URL completo para imagem no Azure Blob Storage udfs.sql	VARCHAR

Mongo DB

-No nosso projeto foram definidos quatro ficheiros em formato JSON que representam a estrutura dos principais dados do sistema. Cada um deles funciona como um modelo lógico da coleção correspondente em MongoDB, fazendo com que especifique campos, tipos de dados, objetos e listas, bem como a informação do carro, do mecânico, códigos de erro, recomendações e anexos.

-A partir destes modelos, criamos as coleções no MongoDB, onde cada documento armazenado segue essa estrutura, fazendo com que o modelo orientado a documentos para concentrar, num único registo, toda a informação relevante de uma intervenção.

-A aplicação só interage com o Django, que valida e estrutura todos os dados através dos models antes de os persistir no MongoDB. Assim, ao criar ordens de serviço, diagnósticos ou recomendações, o Django gera documentos JSON conforme os modelos definidos e garante consistência com a estrutura inicial.

Os ficheiros que usamos:

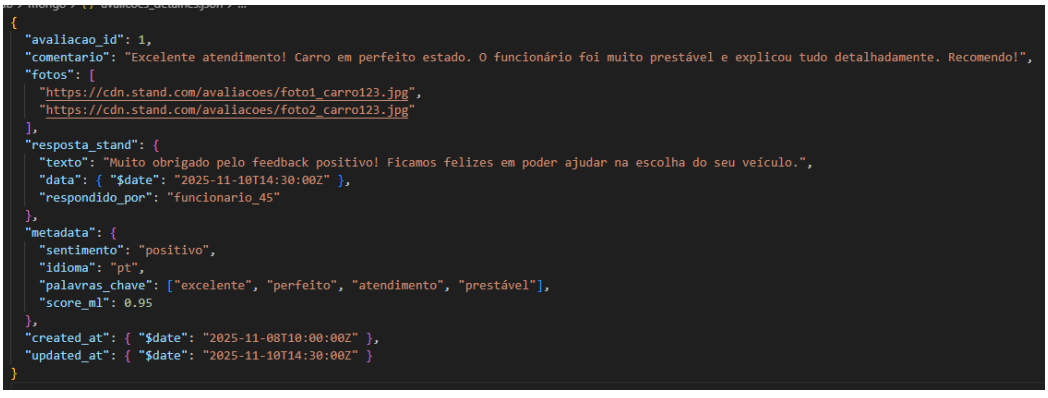
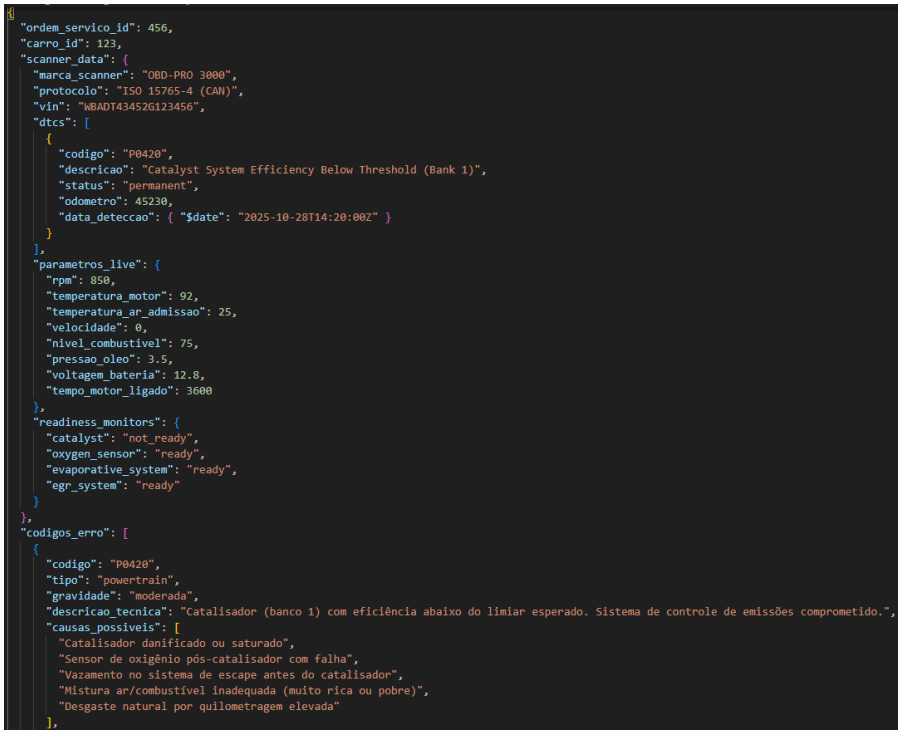


Figura 26-avalicoes_detalhes.json

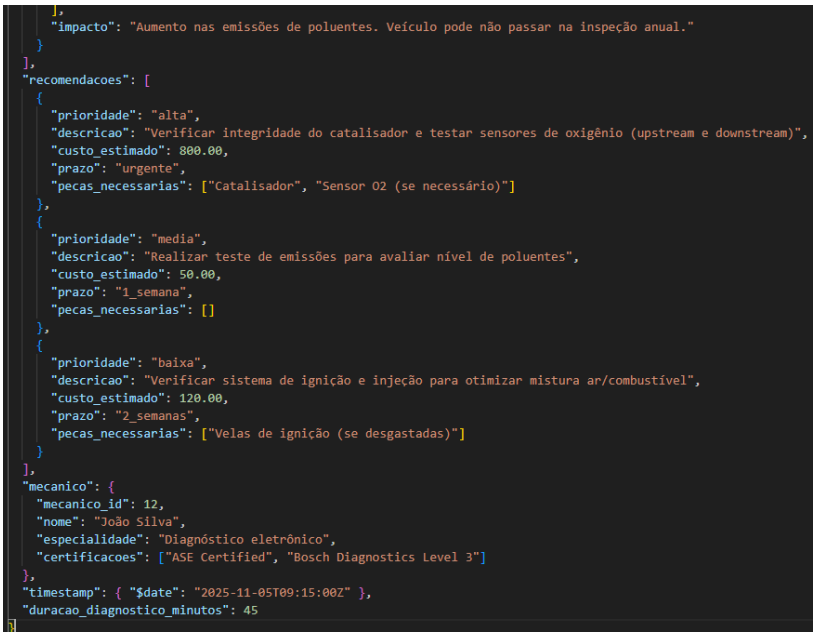
No nosso Sistema do MongoDB, armazena feedback detalhado de clientes sobre serviços.



```
{
  "ordem_servico_id": 456,
  "carro_id": 123,
  "scanner_data": {
    "marca_scanner": "OBD-PRO 3000",
    "protocolo": "ISO 15765-4 (CAN)",
    "vin": "WBADT43452G123456",
    "dtcs": [
      {
        "codigo": "P0420",
        "descricao": "Catalyst System Efficiency Below Threshold (Bank 1)",
        "status": "permanent",
        "odometro": 45230,
        "data_deteccao": { "$date": "2025-10-28T14:20:00Z" }
      }
    ],
    "parametros_live": {
      "rpm": 850,
      "temperatura_motor": 92,
      "temperatura_ar_admissao": 25,
      "velocidade": 0,
      "nivel_combustivel": 75,
      "pressao_oleo": 3.5,
      "voltagem_bateria": 12.8,
      "tempo_motor_ligado": 3600
    },
    "readiness_monitors": {
      "catalyst": "not_ready",
      "oxygen_sensor": "ready",
      "evaporative_system": "ready",
      "egr_system": "ready"
    }
  },
  "codigos_erro": [
    {
      "codigo": "P0420",
      "tipo": "powertrain",
      "gravidade": "moderada",
      "descricao_tecnica": "Catalisador (banco 1) com eficiência abaixo do limiar esperado. Sistema de controle de emissões comprometido.",
      "causas_possiveis": [
        "Catalisador danificado ou saturado",
        "Sensor de oxigênio pós-catalisador com falha",
        "Vazamento no sistema de escape antes do catalisador",
        "Mistura ar/combustível inadequada (muito rica ou pobre)",
        "Desgaste natural por quilometragem elevada"
      ],
```


Figura 27-diagnosticos_oficina.json

No nosso sistema, regista dados OBD, códigos de erro e parâmetros live de diagnósticos.



```
      ],
      "impacto": "Aumento nas emissões de poluentes. Veículo pode não passar na inspeção anual."
    }
  ],
  "recomendacoes": [
    {
      "prioridade": "alta",
      "descricao": "Verificar integridade do catalisador e testar sensores de oxigênio (upstream e downstream)",
      "custo_estimado": 800.00,
      "prazo": "urgente",
      "pecas_necessarias": ["Catalisador", "Sensor O2 (se necessário)"]
    },
    {
      "prioridade": "media",
      "descricao": "Realizar teste de emissões para avaliar nível de poluentes",
      "custo_estimado": 50.00,
      "prazo": "1_semana",
      "pecas_necessarias": []
    },
    {
      "prioridade": "baixa",
      "descricao": "Verificar sistema de ignição e injeção para otimizar mistura ar/combustível",
      "custo_estimado": 120.00,
      "prazo": "2_semanas",
      "pecas_necessarias": ["Velas de ignição (se desgastadas)"]
    }
  ],
  "mecanico": {
    "mecanico_id": 12,
    "nome": "João Silva",
    "especialidade": "Diagnóstico eletrônico",
    "certificacoes": ["ASE Certified", "Bosch Diagnostics Level 3"]
  },
  "timestamp": { "$date": "2025-11-05T09:15:00Z" },
  "duracao_diagnostico_minutos": 45
}
```


Figura 28- diagnosticos_oficina.json (continuação)

No nosso sistema, inclui recomendações, mecânico e duração do diagnóstico.

```
{
  "evento_id": "evt_2025111310450001",
  "tipo": "venda.criada",
  "payload": {
    "venda_id": 789,
    "cliente_id": 123,
    "cliente_nome": "Maria Santos",
    "carro_id": 456,
    "carro_modelo": "BMW X5 2023",
    "preco_final": 43500.00,
    "funcionario_id": 567,
    "stand_id": 1,
    "timestamp": "2025-11-13T10:45:00Z"
  },
  "status": "processado",
  "retry_count": 0,
  "max_retries": 3,
  "proxima_tentativa": null,
  "destinos": [
    {
      "sistema": "ERP",
      "url": "https://erp.stand.com/api/vendas",
      "status": "sucesso",
      "http_status": 201,
      "response_body": { "erp_venda_id": "ERP-789-2025" },
      "processado_em": { "$date": "2025-11-13T10:45:05Z" },
      "tempo_resposta_ms": 234
    },
    {
      "sistema": "BI",
      "url": "https://bi.stand.com/api/eventos",
      "status": "sucesso",
      "http_status": 200,
      "response_body": { "bi_evento_id": "BI-20251113-001" },
      "processado_em": { "$date": "2025-11-13T10:45:08Z" },
      "tempo_resposta_ms": 156
    }
  ],
  "created_at": { "$date": "2025-11-13T10:45:00Z" },
  "updated_at": { "$date": "2025-11-13T10:45:08Z" }
}
```


Figura 29-eventos_integracao.json

No nosso sistema, regista eventos de vendas, processos e integrações com APIs externas.


```
{
  "carro_id": 123,
  "ordem_servico_id": 456,
  "descricao": "Troca de óleo e filtros - Manutenção preventiva dos 45.000 km",
  "data": { "$date": "2025-11-05T09:00:00Z" },
  "mecanico": {
    "mecanico_id": 12,
    "nome": "João Silva",
    "especialidade": "Manutenção preventiva"
  },
  "pecas_utilizadas": [
    {
      "peca_id": 101,
      "nome": "Óleo Sintético 5W30",
      "quantidade": 5,
      "codigo": "OL5W30-001",
      "preco_unitario": 15.50
    },
    {
      "peca_id": 102,
      "nome": "Filtro de Óleo",
      "quantidade": 1,
      "codigo": "FO-BMW-2024",
      "preco_unitario": 22.00
    },
    {
      "peca_id": 103,
      "nome": "Filtro de Ar",
      "quantidade": 1,
      "codigo": "FA-BMW-2024",
      "preco_unitario": 18.00
    }
  ],
  "diagnostico": {
    "descricao": "Veículo em bom estado geral. Óleo antigo apresentava cor escura mas sem contaminação. Recomendado verificar pastilhas de freio nos próximos
    "quilometragem": 45000,
    "nivel_urgencia": "baixo",
    "observacoes": "Cliente solicitou lavagem de motor. Executado sem custo adicional."
  },
```


Figura 30-historico_manutencao.json

No nosso sistema, armazena histórico de manutenções com peças e custos.



Figura 31 - historico_manutencao.json(continuação)

No nosso sistema, inclui anexos, diagnóstico e próxima revisão.

Frontend

Tipos de Utilizadores da Aplicação:
A aplicação **Auto Care** possui três tipos principais de utilizadores com diferentes níveis de permissões:

- **Administrador (Admin)**: Utilizador com permissões totais no sistema, identificado através de user.is_superuser. Tem acesso a todas as funcionalidades de gestão, incluindo eliminação de registos, edição de dados sensíveis e acesso completo às operações da oficina.
- **Funcionário/Mecânico/Gerente (Staff)**: Utilizador com permissões intermédias, identificado através de user.is_staff. Pode gerir carros, clientes, reservas, vendas e ordens de serviço, mas com algumas restrições em comparação com o administrador.
- **Cliente**: Utilizador com permissões limitadas. Pode visualizar carros disponíveis, as suas próprias reservas e ordens de serviço, deixar avaliações e aceder a informações relacionadas com as suas transações.

Estrutura da Aplicação Web

A aplicação está organizada em páginas especializadas que implementam diferentes funcionalidades do sistema.

Páginas de Autenticação

- login.html: Página de autenticação onde o utilizador insere email completo como username
    - Requisitos: Validação de credenciais, redirecionamento pós-login
    - Responsável: Tiago

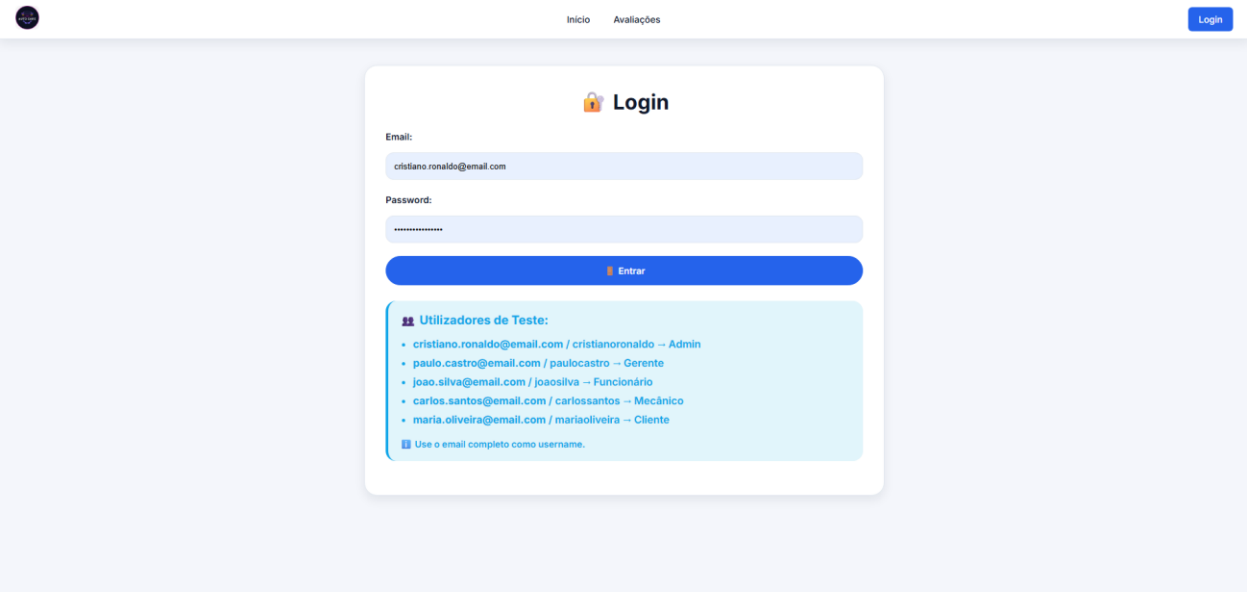


Figura 32 - Login

- base.html: Template base com estrutura comum (navegação, estilos, scripts)
    - Requisitos: Layout responsivo, menu de navegação dinâmico baseado em permissões
    - Responsável: Lucas

Páginas Principais

- home.html: Página inicial que apresenta saudação personalizada e perfil do utilizador
    - Requisitos: Exibição de nome de utilizador e tipo de conta (Admin/Funcionário/Cliente)
    - Responsável: Guilherme



Figura 33 – Início

- carros.html: Listagem e gestão de veículos do stand
    - Requisitos: Filtros de pesquisa, exibição de ano/preço/status/estado, ações de edição para staff
    - Responsável: Lucas

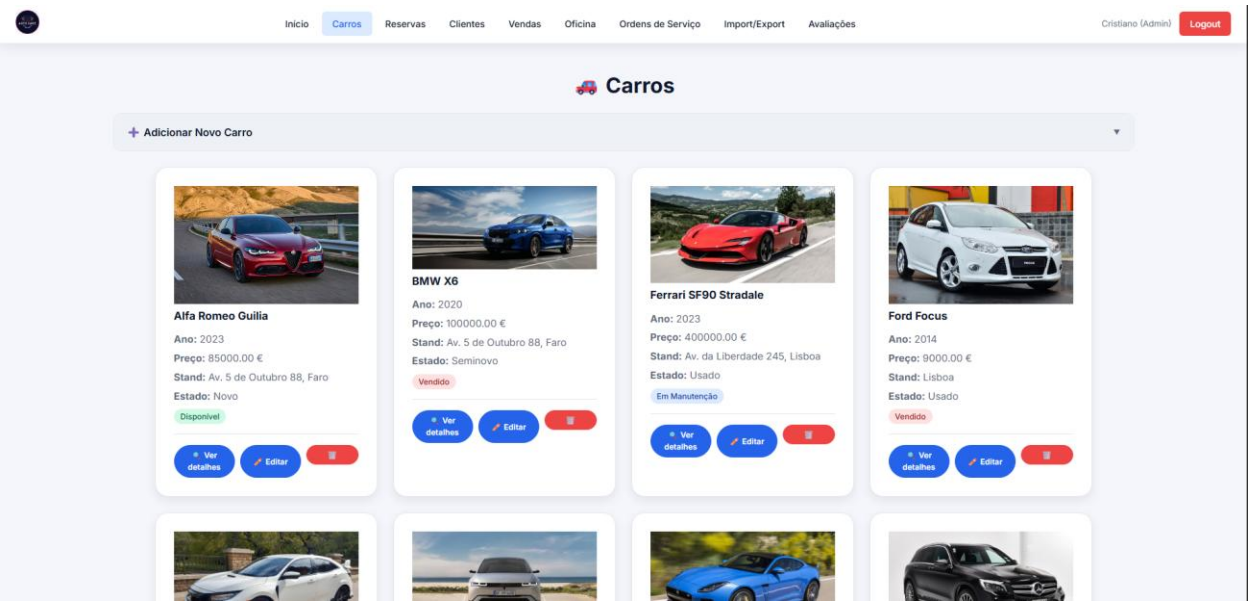


Figura 34 – Carros

- detalhes_carro.html: Informações detalhadas de um veículo específico
    - Requisitos: Visualização completa de especificações, histórico, possibilidade de reserva
    - Responsável: Guilherme

Páginas de Gestão de Clientes

- clientes.html: Gestão de informações de clientes
    - Requisitos: CRUD de clientes (email, telefone, NIF), controlo de permissões para edição
    - Responsável: Tiago

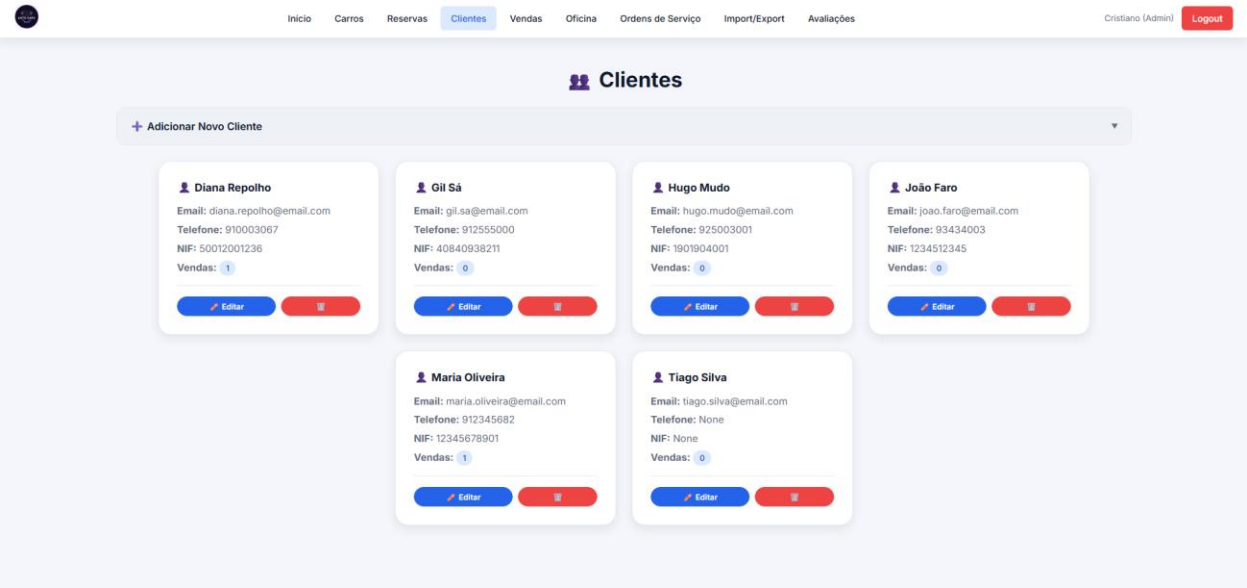


Figura 35 - -Clientes

- editar_cliente.html: Formulário de edição de dados de cliente
    - Requisitos: Validação de dados, atualização de registos
    - Responsável: Lucas

Páginas de Reservas e Vendas

- reservas.html: Gestão de reservas de veículos
    - Requisitos: Listagem de reservas por cliente/carro/data, controlo de estados
    - Responsável: Guilherme

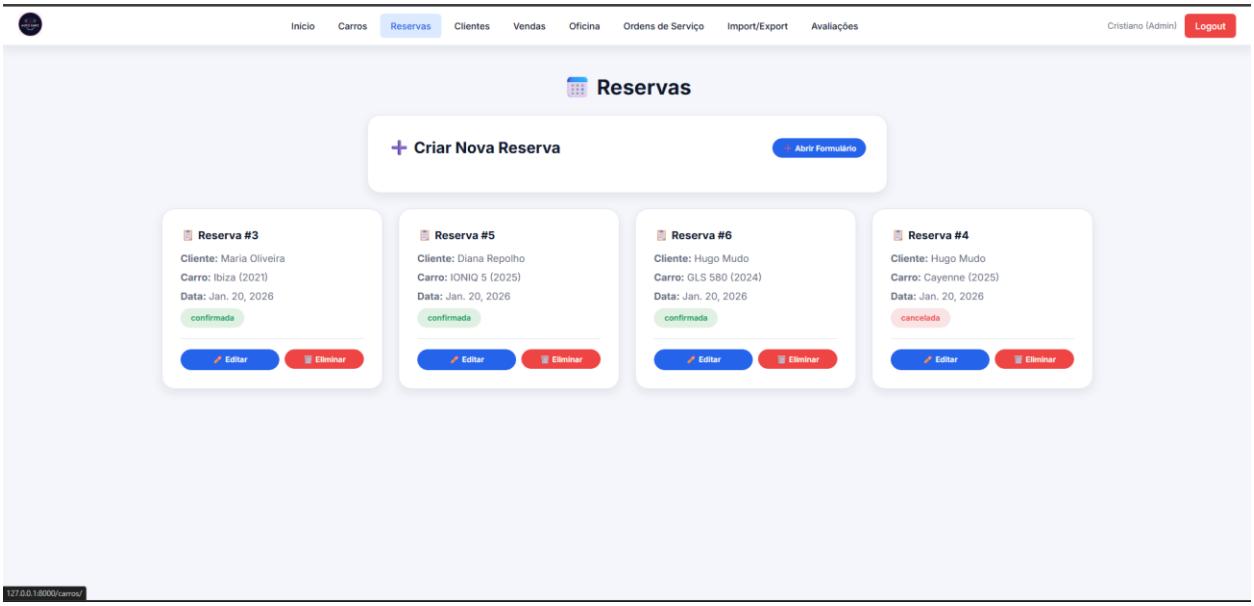


Figura 36 – Reservas

- editar_reserva.html: Edição de informações de reserva
    - Requisitos: Alteração de estado, cancelamento, confirmação
    - Responsável: Tiago
- vendas.html: Registo e consulta de vendas concluídas
    - Requisitos: Visualização de cliente/carro/data/valor/funcionário, permissões para registo
    - Responsável: Lucas

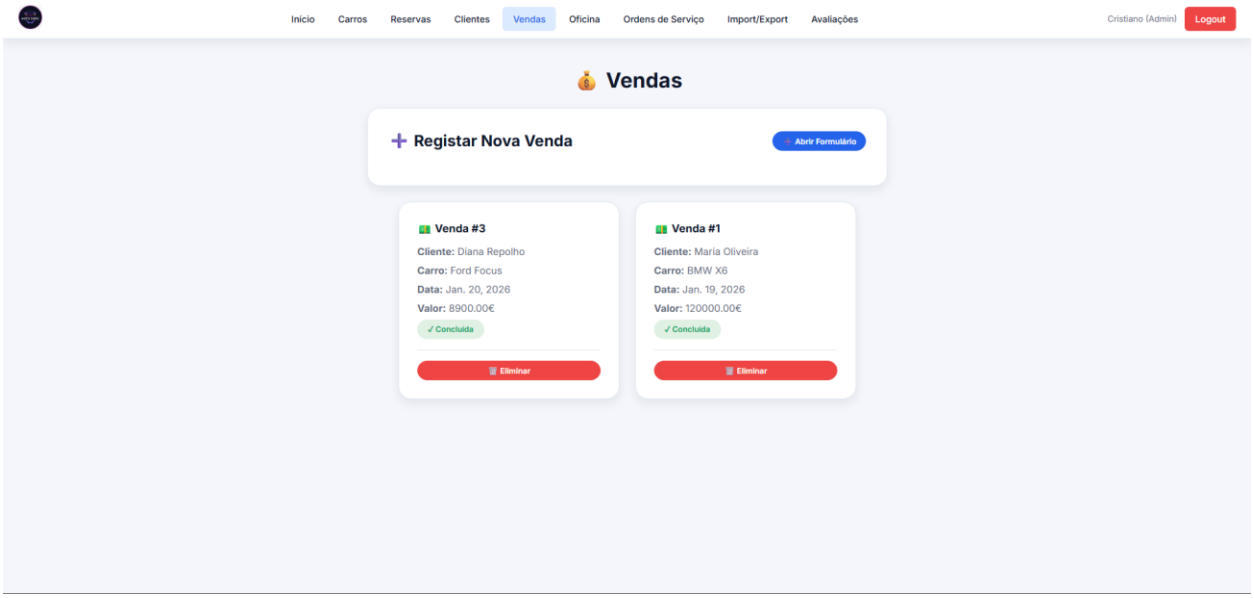


Figura 37 – Vendas

- pagamentos.html: Gestão de pagamentos e parcelas
    - Requisitos: Tabela com ID venda, valor, método de pagamento, número de parcela
    - Responsável: Guilherme

Páginas de Oficina e Serviços

- oficina.html: Gestão completa de manutenções, diagnósticos e avaliações usando MongoDB
    - Requisitos: CRUD de manutenções com peças/custos/tempo, diagnósticos OBD-II com códigos DTC, eventos de integração, controlo de permissões por tipo de utilizador
    - Responsável: Tiago

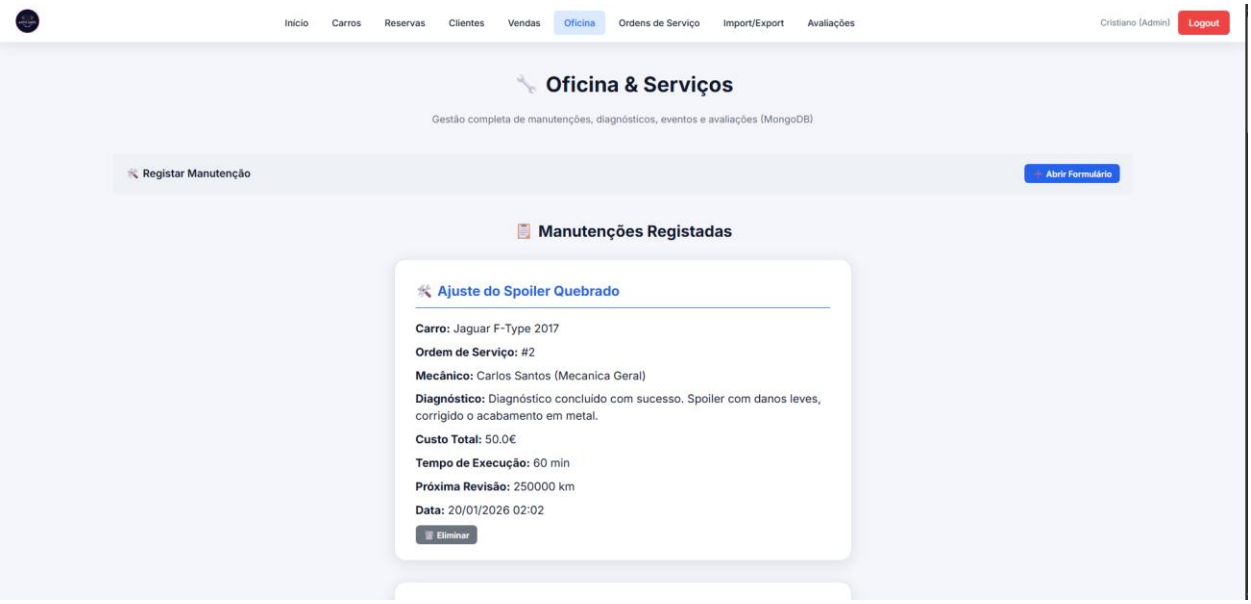


Figura 38 – Oficina parte 1

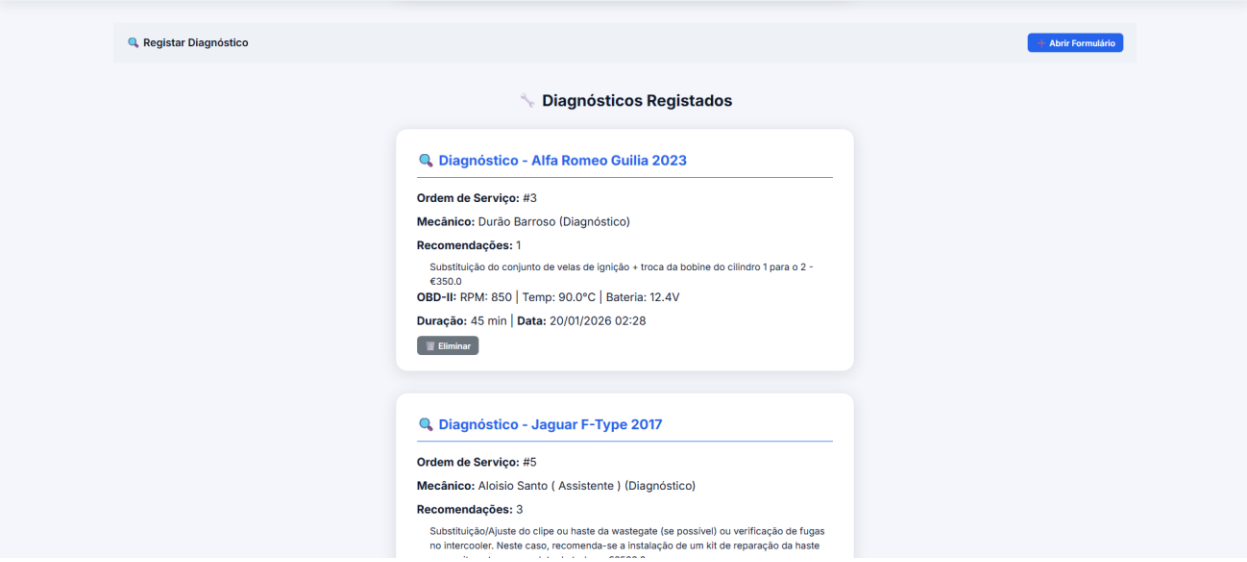


Figura 39- Oficina parte 2

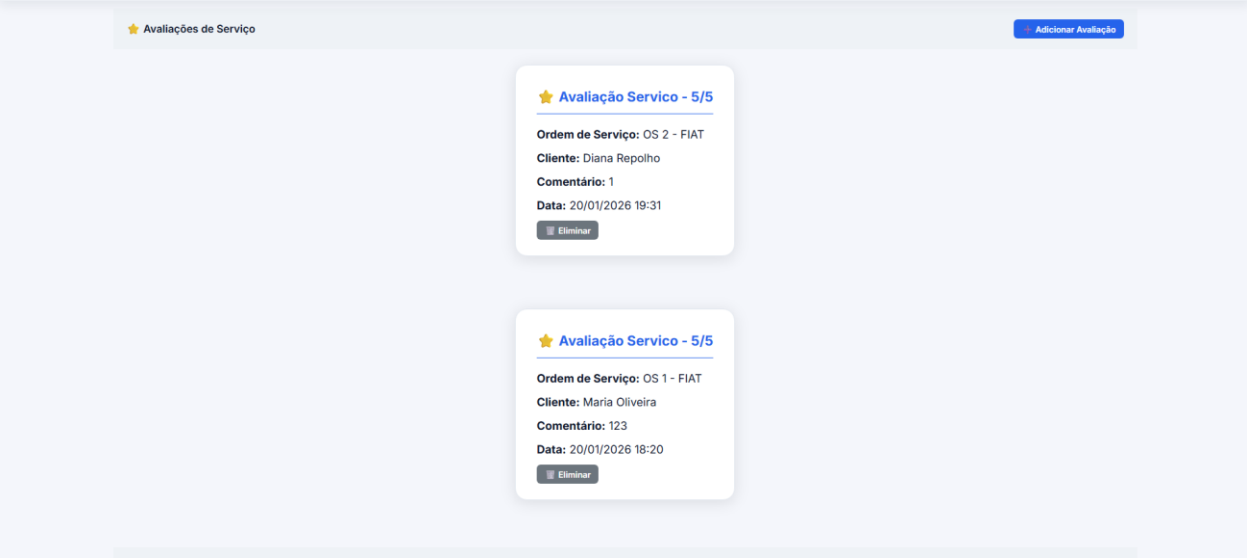


Figura 40 - Oficina parte 3

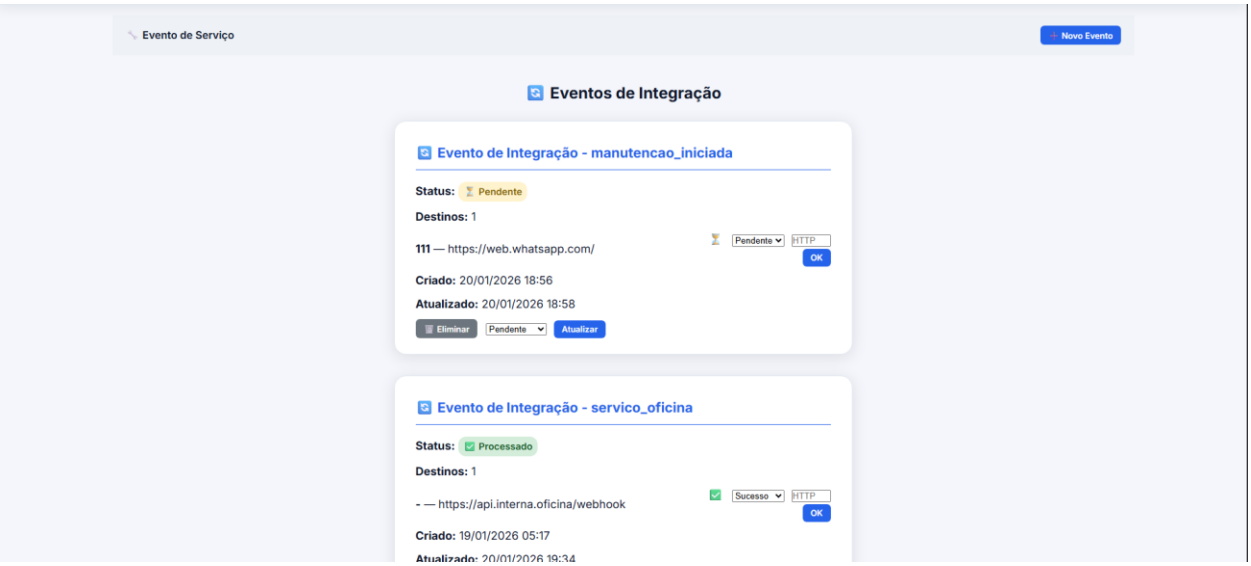


Figura 41 - Oficina parte 4

- ordens_servico.html: Listagem e gestão de ordens de serviço
    - Requisitos: Diferentes visualizações (ativas/atrasadas/concluídas/cliente), priorização, atribuição de mecânicos, cálculo de dias de atraso
    - Responsável: Lucas

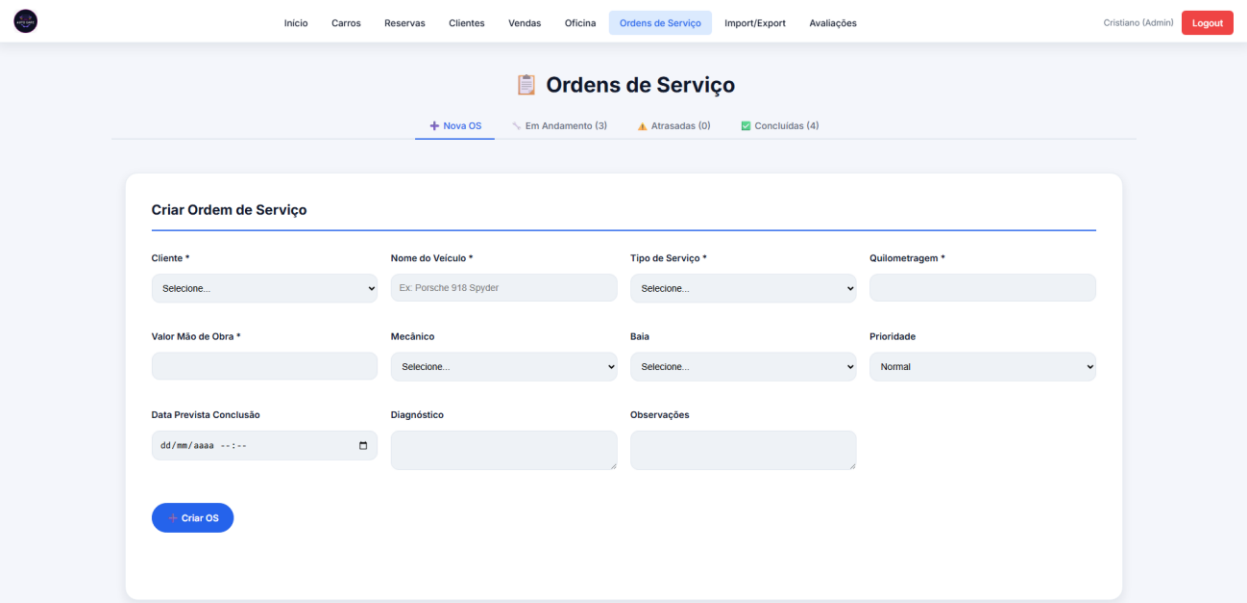


Figura 42 - Ordens de Serviço

- detalhes_ordem_servico.html: Informações detalhadas de uma ordem de serviço
    - Requisitos: Exibição completa de serviços, peças, valores, histórico de estados
    - Responsável: Guilherme

Páginas de Recursos Humanos

- editar_funcionario.html: Gestão de dados de funcionários
    - Requisitos: Formulário com validação de informações de staff
    - Responsável: Tiago
- editar_mecanico.html: Gestão de informações de mecânicos
    - Requisitos: Campos específicos incluindo especialidade, atribuição a ordens de serviço
    - Responsável: Lucas
- editar_fornecedor.html: Gestão de fornecedores de peças
    - Requisitos: CRUD de fornecedores para gestão de inventário

- Responsável: Guilherme

Páginas de Sistema

- avaliacoes.html: Sistema de avaliações de clientes
    - Requisitos: Classificação por estrelas, comentários, tipo de avaliação, permissões de eliminação
    - Responsável: Tiago

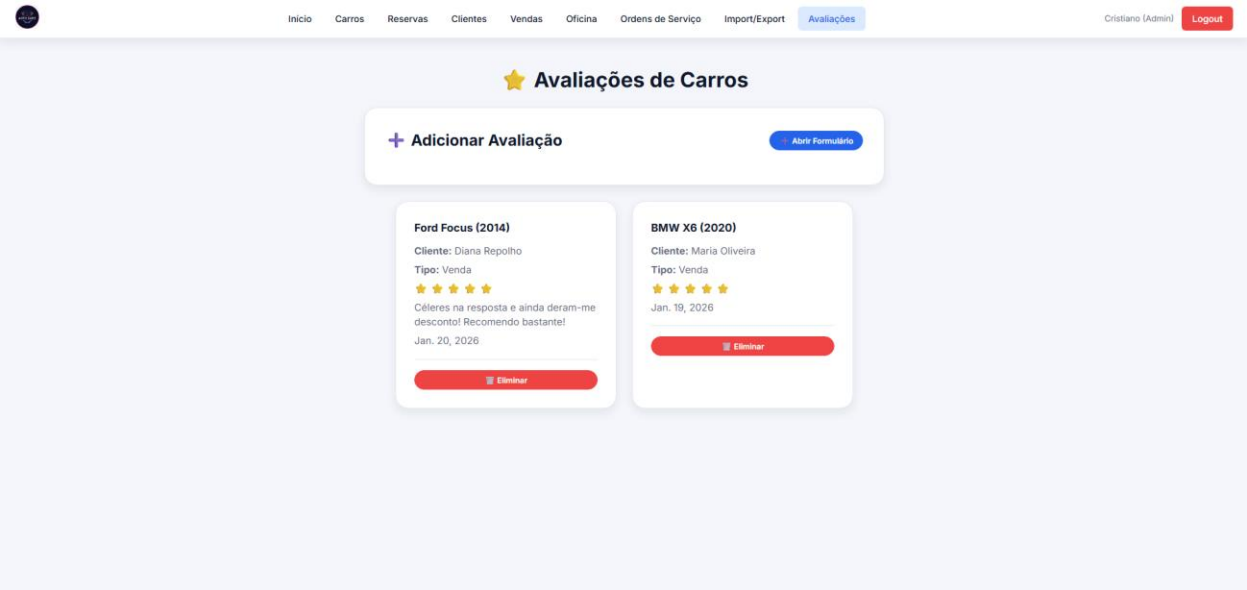


Figura 43 – Avaliações

- import_export.html: Funcionalidade de importação/exportação de dados em JSON
    - Requisitos: Export de dados do sistema, upload e validação de ficheiros JSON com formato específico
    - Responsável: Lucas

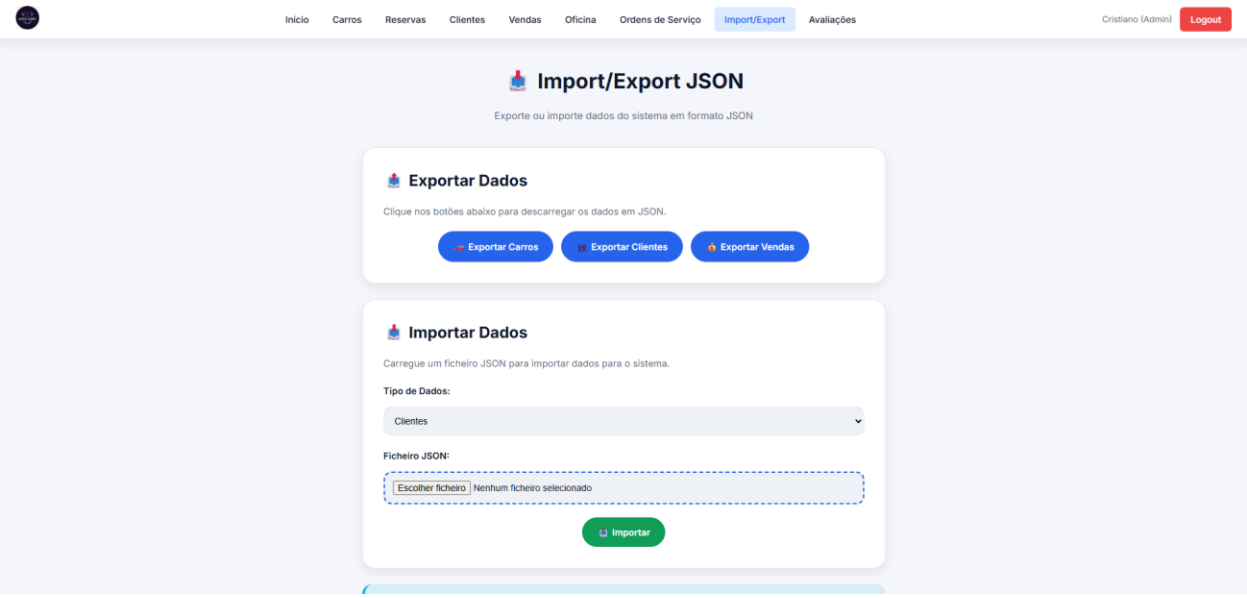


Figura 44 - Import/Export